

EDS CodeTantra Lab Course

Name: Akshara Rodge

Batch: CH-2

PRN: 202501050040

Practical-4

4.1 Practice Assignments

4.1.1. Pandas series creation

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean()` operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

Program:

```
import pandas as pd
```

```
# Take inputs from the user to create a list of numbers
```

```
numbers = list(map(int, input().split()))
```

```
# Create a Pandas series from the list of numbers
```

```
series=pd.Series(numbers)
```

```
# Grouping by even and odd numbers and calculating the mean
```

```
grouped = series.groupby(series%2==0).mean()
```

```
# Display the mean of even and odd numbers with labels
```

```
grouped.index = ['Even' if is_even else 'Odd' for is_even in  
grouped.index]
```

```
print("Mean of even and odd numbers:")
```

```
print(grouped)
```

Screenshot:

The screenshot displays the CodeTANTRA web application. On the left, the problem statement for '4.1.1. Pandas - series creation and manipulation' is visible, including input/output formats and sample test cases. The main editor area on the right contains the following Python code:

```
1 import pandas as pd
2
3 # Take inputs from the user to create a list of numbers
4 numbers = list(map(int, input().split()))
5
6 # Create a Pandas series from the list of numbers
7 series = pd.Series(numbers)
8 # Grouping by even and odd numbers and calculating the mean
9 grouped = series.groupby(series%2==0).mean()
10
11 # Display the mean of even and odd numbers with labels
12 grouped.index = ['Even' if is_even else 'Odd' for is_even in
13 grouped.index]
14 print("Mean of even and odd numbers:")
15 print(grouped)
```

Below the code editor, the test results are shown: '3 out of 3 shown test case(s) passed' and '3 out of 3 hidden test case(s) passed'. A detailed view of 'Test case 1' is expanded, showing the expected and actual outputs for the input '1 2 3 4 5 6 7 8 9 10'.

Expected output	Actual output
1 2 3 4 5 6 7 8 9 10	1 2 3 4 5 6 7 8 9 10
Mean of even and odd numbers:	Mean of even and odd numbers:
Odd 5.0	Odd 5.0
Even 6.0	Even 6.0
dtype: float64	dtype: float64

4.1.2 Dictionary to dataframe

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Delete a column:

- Remove the **Age** column.
- Display the DataFrame after deleting the column.

Program:

```
import pandas as pd
```

```
# Provided dictionary of lists
```

```
data = {  
    'Name': ['Alice', 'Bob', 'Charlie'],  
    'Age': [25, 30, 35],  
}
```

```
# Convert the dictionary to a DataFrame
```

```
df = pd.DataFrame(data)
```

```
# Display the original DataFrame
```

```
print("Original DataFrame:")
```

```
print(df)
```

```
# Adding a new row
```

```
name = input("New name: ")
```

```
age = int(input("New age: "))
```

```
df = df.append({"Name":name,"Age":age},ignore_index=True)
```

```
# Display the DataFrame after adding a new row
```

```
print("After adding a row:\n",df)
```

```
# Modifying a row
```

```
idx = int(input("Index of row to modify: "))
```

```
age = int(input("New age: "))
```

```
df.loc[idx,"Age"] = age
```

```
# Display the DataFrame after modifying a row
```

```
print("After modifying a row:")
```

```
print(df)
```

```
# Deleting a row
```

```
idx = int(input("Index of row to delete: "))
```

```
df = df.drop(idx).reset_index(drop=True)
```

```
# Display the DataFrame after deleting a row
```

```
print("After deleting a row:")
```

```
print(df)
```

```
# Adding a new column
```

```
g = input("Enter genders separated by space: ").split(" ")
```

```
df["Gender"] = g
```

```
# Display the DataFrame after adding a new column
```

```
print("After adding a new column:")
```

```
print(df)
```

```
# Modifying a column
```

```
df["Name"] = df["Name"].str.upper()
```

Display the DataFrame after modifying a column

```
print("After modifying a column:")
```

```
print(df)
```

Deleting a column

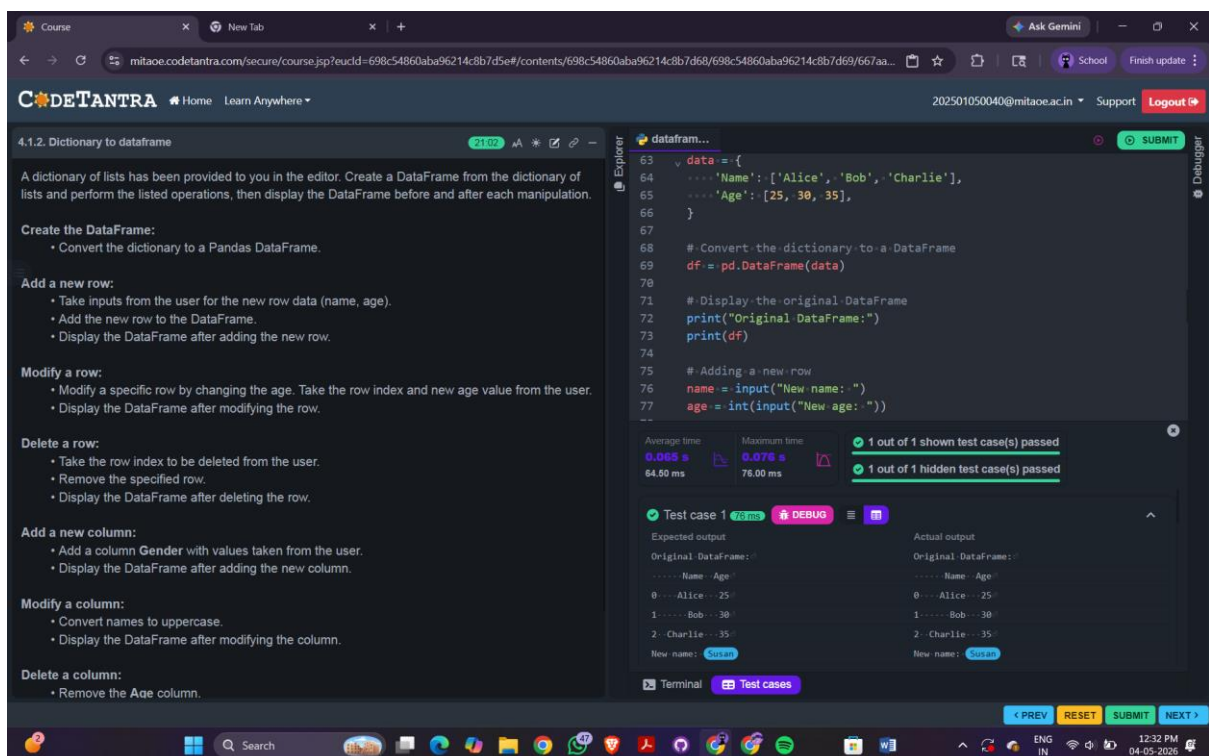
```
df = df.drop(columns=["Age"])
```

Display the DataFrame after deleting a column

```
print("After deleting a column:")
```

```
print(df)
```

Screenshot:



4.1.3 Student information

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Note:

Refer to the displayed test cases for better understanding.

Program:

```
import pandas as pd

# Read the text file into a DataFrame
file = input()

data = pd.read_csv(file, sep="\s+", header=None, names=["Name",
"Age", "Grade"])

# write your code here..

print("First five rows:")

print(data.head())

average_age= round(data["Age"].mean(),2)

print(f"Average age: {average_age}")

filtered_students = data[data["Grade"] <= "B"]

print("Students with a grade up to B")

print(filtered_students)
```

Screenshot:

Course

mitaoe.codetanttra.com/secure/course.jsp?eucld=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d5e/667cd...

Ask Gemini

Course

202501050040@mitaoe.ac.in Support Logout

4.1.3. Student Information

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Note:
Refer to the displayed test cases for better understanding.

Sample Test Cases

studentin...

```

1 import pandas as pd
2
3 # Read the text file into a DataFrame
4 file = input()
5 data = pd.read_csv(file, sep='\t', header=None, names=["Name", "Age",
6 "Grade"])
7
8 # write your code here..
9 print("First five rows:")
10 print(data.head())
11
12 average_age = round(data["Age"].mean(), 2)
13 print(f"Average age: {average_age}")
14

```

Average time: 0.033 s
Maximum time: 0.047 s
32.80 ms 47.00 ms

1 out of 1 shown test case(s) passed
1 out of 1 hidden test case(s) passed

Test case 1

Expected output

studentdata.txt

First five rows:

	Name	Age	Grade
0	John	25	A
1	Alin	22	B
2	Emma	24	A

Actual output

studentdata.txt

First five rows:

	Name	Age	Grade
0	John	25	A
1	Alin	22	B
2	Emma	24	A

Terminal Test cases

12:32 PM 04-05-2026

4.2 Lab Assignments

4.2.1 Month with the highest total sales

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: `Date`, `Product`, `Quantity`, `Price`, and `City`.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Program:

```
import pandas as pd
```

```
# Prompt the user for the file name
```

```
file_name = input()
```

```
# Load the data
```

```
df = pd.read_csv(file_name)
```

```
df["Date"] = pd.to_datetime(df["Date"])
```

```
df["Month"] = df["Date"].dt.to_period("M")
```

```
df["Total Sales"] = df["Quantity"] * df["Price"]
```

```
monthly_sales = df.groupby("Month")["Total Sales"].sum()
```

```
# Find the month with the highest total sales
```

```
best_month = monthly_sales.idxmax()
```

```
highest_sales = monthly_sales.max()
```

```
print(f"Best month: {best_month}")
```

```
print(f"Total sales: ${highest_sales:.2f}")
```

Screenshot:

Course

New Tab

mitaoe.codetantra.com/secure/course.jsp?euid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d5e/698c54860aba96214c8b7d5e/67860...

Ask Gemini

School

Finish update

CODETANTRA

Home

Learn Anywhere

202501050040@mitaoe.ac.in

Support

Logout

4.2.1. Month with the Highest Total Sales

04:10

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```

1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8 df["Date"] = pd.to_datetime(df["Date"])
9 df["Month"] = df["Date"].dt.to_period("M")
10 df["Total Sales"] = df["Quantity"] * df["Price"]
11 monthly_sales = df.groupby("Month")["Total Sales"].sum()
12 # Find the month with the highest total sales
13 best_month = monthly_sales.idxmax()
14 highest_sales = monthly_sales.max()
15

```

Average time: 0.017 s

Maximum time: 0.033 s

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

Expected output: sales_data.csv

Actual output: sales_data.csv

Best month: 2025-01

Total sales: \$1210.00

Best month: 2025-01

Total sales: \$1210.00

Terminal

Test cases

PREV

RESET

SUBMIT

NEXT

12:32 PM

04-05-2026

4.2.2 Best selling product

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Program:

```

import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)

product_sales=df.groupby("Product")["Quantity"].sum()

best_product= product_sales.idxmax()

highest_quantity=product_sales.max()

# Find the product with the highest total quantity sold

# Display the result
print(f"Best selling product: {best_product}")

print(f"Total quantity sold: {highest_quantity}")

```

Screenshot:

The screenshot shows a web-based IDE interface with a dark theme. The main editor displays a Python script for finding the best-selling product. The script prompts the user for a file name, loads the data, and displays the results. The output shows 'Product A' as the best-selling product with a total quantity sold of 23.

4.2.2. Best Selling Product

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	10	10	New York
2025-01-01	Product A	15	15	Los Angeles
2025-01-01	Product A	7	20	New York
2025-01-01	Product C	6	30	Chicago
2025-01-01	Product C	7	15	Chicago
2025-01-01	Product C	6	30	Los Angeles
2025-01-01	Product C	6	30	New York
2025-01-01	Product A	5	15	Los Angeles
2025-01-01	Product A	1	30	Chicago
2025-01-01	Product C	10	30	Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

Test Case	Expected output	Actual output
Test case 1	sales_data.csv Best selling product: Product A Total quantity sold: 23	sales_data.csv Best selling product: Product A Total quantity sold: 23

4.2.3 City that sold the most products

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: `Date`, `Product`, `Quantity`, `Price`, and `City`.
- Group the data by `City` and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Program:

```
import pandas as pd
```

```
# Prompt the user for the file name
```

```
file_name = input()
```

```
# Load the data
```

```
df = pd.read_csv(file_name)
```

```
city_sales=df.groupby("City")["Quantity"].sum()
```

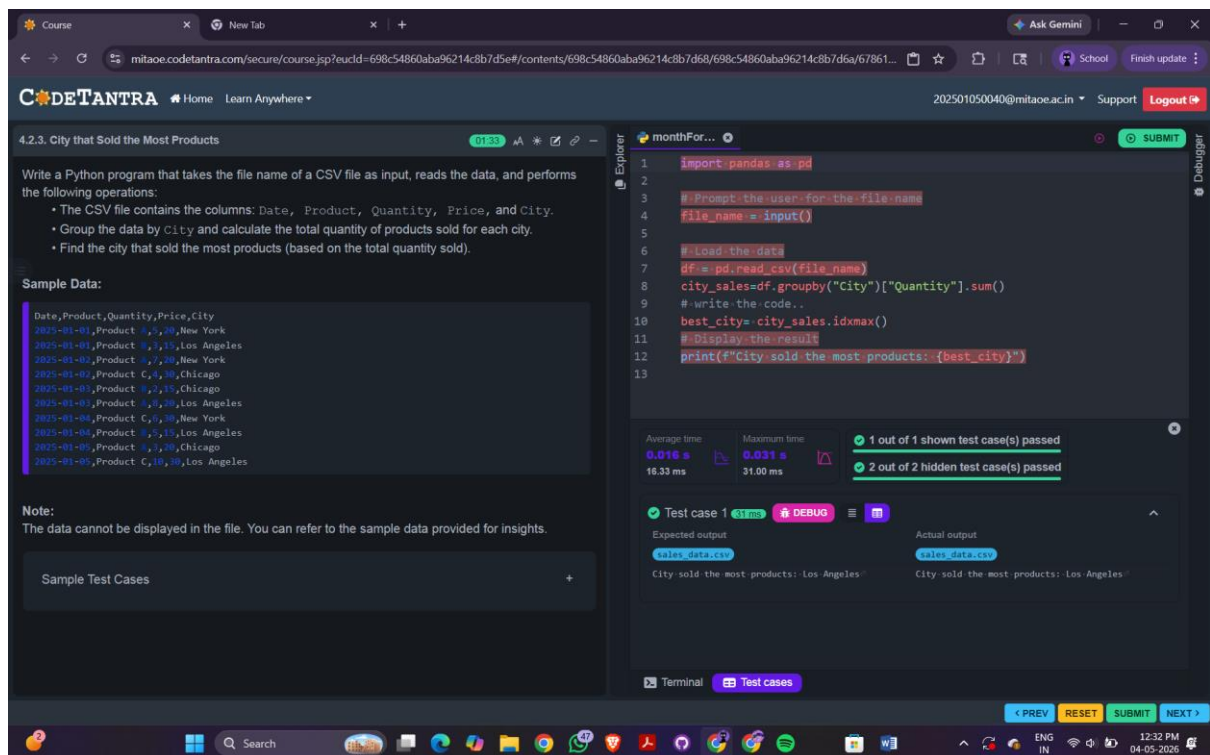
```
# write the code..
```

```
best_city= city_sales.idxmax()
```

```
# Display the result
```

```
print(f"City sold the most products: {best_city}")
```

Screenshot:



4.2.4 Most frequently sold product pair

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Explanation:

Transactions:

- **2025-01-01:** Product A, Product B
- **2025-01-02:** Product A, Product C
- **2025-01-03:** Product B, Product A
- **2025-01-04:** Product C, Product B
- **2025-01-05:** Product A, Product C

Now, let's count how often the pairs of products appear together:

- **Product A and Product B:** Appear in transactions on 2025-01-01 and 2025-01-03.
- **Product A and Product C:** Appear in transactions on 2025-01-02 and 2025-01-05.
- **Product B and Product C:** Appears in transactions on 2025-01-04.

Most Frequent Product Combinations:

- **Product A and Product B** (2 times)
- **Product A and Product C** (2 times)

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Program:

```
import pandas as pd
```

```
from itertools import combinations
```

```
from collections import Counter
```

```
# Prompt user to input the file name
```

```
file_name = input()
```

```
# Read data from the specified CSV file
```

```
df = pd.read_csv(file_name)
```

```
grouped = df.groupby('Date')['Product'].apply(list)
```

```
pairs = []
```

```
for products in grouped:
```

```
    if len(products) > 1:
```

```
        products = sorted(products)
```

```
        for i in range(len(products)):
```

```
            for j in range(i+1, len(products)):
```

```
                pairs.append((products[i], products[j]))
```

```
pairs_df = pd.DataFrame(pairs, columns=['Product1',  
'Product2'])
```

```
pair_counts =
```

```
pairs_df.value_counts().reset_index(name='Count')
```

```
max_count = pair_counts['Count'].max()
```

```
most_frequent_pairs = pair_counts[pair_counts['Count'] ==  
max_count]
```

```
for _, row in most_frequent_pairs.iterrows():
```

```
print(f"{row['Product1']} and {row['Product2']}:  
{row['Count']} times")
```

Screenshot:

The screenshot shows a web-based Python IDE interface. On the left, the problem statement for '4.2.4. Most Frequently Sold Product Pairs' is displayed, along with a sample CSV data and an explanation of the expected output. The main editor shows the following Python code:

```
1 import pandas as pd
2 from itertools import combinations
3 from collections import Counter
4
5 # Prompt user to input the file name
6 file_name = input()
7
8 # Read data from the specified CSV file
9 df = pd.read_csv(file_name)
10
11 grouped = df.groupby('Date')['Product'].apply(list)
12
13 pairs = []
14
15 for products in grouped:
```

Below the code, the test results are shown: '1 out of 1 shown test case(s) passed' and '2 out of 2 hidden test case(s) passed'. A test case comparison shows the expected output 'sales_data.csv' and the actual output 'sales_data.csv'.

4.2.5 Titanic dataset analysis

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (`.mode()`).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 1759
9,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O 2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C1
23,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11
.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

```

Note: Refer to the visible test case for better reference.

Program:

```
import pandas as pd
```

```
import numpy as np
```

Load the Titanic dataset

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
head = data.head()
```

```
print(head)
```

2. Last 5 rows

```
tail = data.tail()
```

```
print(tail)
```

3. Shape

```
print(data.shape)
```

4. Full .info() output with memory usage

```
print(data.info())
```

5. Basic stats

```
print(data.describe())
```

6. Missing values

```
print(data.isnull().sum())
```

7. Fill missing 'Age' with median

```
data['Age'].fillna(data['Age'].median(), inplace=True)
```

8. Fill missing 'Embarked' with mode

```
data['Embarked'].fillna(data['Embarked'].mode()[0],  
inplace=True)
```

9. Drop 'Cabin'

```
data.drop('Cabin', axis=1, inplace=True)
```

10. Create 'FamilySize'

```
data['FamilySize'] = data['SibSp'] + data['Parch']
```

Screenshot:

4.2.5. Titanic Dataset Analysis and Data Cleaning

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. Brown, Mrs. Brown	male	22	1	0	A/5 21171, 7.25	5.12	S	S
2	1	1	Mr. Brown, Mrs. Brown	female	38	1	0	PC 17599, 71.2833	53.1	C	S
3	1	3	Mr. Brown, Mrs. Brown	female	26	0	0	STON/O2, 3101702, 7.92	5.12	S	S

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1, 0, 3, Mr. Brown, Mrs. Brown, male, 22, 1, 0, A/5 21171, 7.25, S, S
2, 1, 1, Mr. Brown, Mrs. Brown, female, 38, 1, 0, PC 17599, 71.2833, C, S
3, 1, 3, Mr. Brown, Mrs. Brown, female, 26, 0, 0, STON/O2, 3101702, 7.92, S, S
```

Code Editor:

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 head = data.head()
8 print(head)
9
10 # 2. Last 5 rows
11 tail = data.tail()
12 print(tail)
13
14 # 3. Shape
15 print(data.shape)
```

Test Results:

1 out of 1 shown test case(s) passed

4.2.6 Titanic dataset analysis 2

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 1759
9,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C1
23,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11
.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C
```

Note: Refer to the visible test case for better reference.

Program:

```
import pandas as pd
```

```
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
data['FamilySize'] = data['SibSp'] + data['Parch']
```

```
# 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
```

```
data['IsAlone'] = np.where(data['FamilySize'] == 0, 1, 0)
```

```
# 2. Convert 'Sex' to numeric (male: 0, female: 1)
```

```
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
```

3. One-hot encode the 'Embarked' column

```
data = pd.get_dummies(data, columns=['Embarked'],  
prefix='Embarked', drop_first=True)
```

4. Get the mean age of passengers

```
mean_age = data['Age'].mean()  
print(mean_age)
```

5. Get the median fare of passengers

```
median_fare = data['Fare'].median()  
print(median_fare)
```

6. Get the number of passengers by class

```
pc = data['Pclass'].value_counts()  
print(pc)
```

7. Get the number of passengers by gender

```
pg = data['Sex'].value_counts()  
print(pg)
```

8. Get the number of passengers by survival status

```
ps = data['Survived'].value_counts()
print(ps)
```

9. Calculate the survival rate

```
sr = data['Survived'].mean()
print(sr)
```

10. Calculate the survival rate by gender

```
srg = data.groupby('Sex')['Survived'].mean()
print(srg)
```

Screenshot:

The screenshot displays a Jupyter Notebook environment with a dark theme. The left sidebar shows the file explorer with a notebook named 'titanicDat...'. The main area contains a code cell with Python code for data analysis. The code includes imports for pandas and numpy, loading the Titanic dataset, and performing various data manipulations like creating a new column 'IsAlone', converting 'Sex' to numeric, and one-hot encoding 'Embarked'. The output of the code is visible, showing the average time (0.057 s) and maximum time (57.00 ms) for the execution, along with a message indicating that 1 out of 1 shown test case(s) passed. Below the code, there is a table showing the Titanic dataset columns: PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, and Embarked. The table is partially filled with sample data. At the bottom, there is a terminal window showing the command prompt and the output of the code execution.

4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. Brown, Mr. Owen Harris	male	21	1	0	A/5 21171, Z 24	5.42	S	S
2	1	1	Mr. John Bradley (Florence Briggs Thayer)	female	36	1	0	PC 17599, 71 2831, C85	53.1	C	S
3	1	1	Mr. James (Miss. Laina)	female	36	0	0	STON/O2. 3401282, 7 925	5.42	S	S
4	1	1	Mr. Jacques Heath (Lily May Peel)	female	36	1	0	113804, 53 1, C123	5.42	S	S

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1, 0, 3, Mr. Brown, Mr. Owen Harris, male, 21, 1, 0, A/5 21171, Z 24, S, S
2, 1, 1, Mr. John Bradley (Florence Briggs Thayer), female, 36, 1, 0, PC 17599, 71 2831, C85, C
3, 1, 1, Mr. James (Miss. Laina), female, 36, 0, 0, STON/O2. 3401282, 7 925, S, S
4, 1, 1, Mr. Jacques Heath (Lily May Peel), female, 36, 1, 0, 113804, 53 1, C123, S
```

Test case 1 **✓** **0.057 s** **57.00 ms** **1 out of 1 shown test case(s) passed**

Expected output: 29.69911764795882, 14.4542, 3... 491, 1... 216, 2... 184

Actual output: 29.69911764795882, 14.4542, 3... 491, 1... 216, 2... 184

Name: Pclass, dtype: int64

Terminal: **Test cases**

4.2.7 Titanic dataset analysis

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 1759
9,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C1
23,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11
.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

```

Note: Refer to the visible test case for better reference.

Program:

```
import pandas as pd
```

```
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']
data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
data = pd.get_dummies(data, columns=['Embarked'],
drop_first=True)
```

1. Calculate the survival rate by class

```
print(data.groupby('Pclass')['Survived'].mean())
```

2. Calculate the survival rate by embarked location

```
print(data.groupby('Embarked_S')['Survived'].mean())
```

3. Calculate the survival rate by family size

```
print(data.groupby('FamilySize')['Survived'].mean())
```

4. Calculate the survival rate by being alone

```
print(data.groupby('IsAlone')['Survived'].mean())
```

5. Get the average fare by class

```
print(data.groupby('Pclass')['Fare'].mean())
```

6. Get the average age by class

```
print(data.groupby('Pclass')['Age'].mean())
```

7. Get the average age by survival status

```
print(data.groupby('Survived')['Age'].mean())
```

8. Get the average fare by survival status

```
print(data.groupby('Survived')['Fare'].mean())
```

9. Get the number of survivors by class

```
print(data[data['Survived'] == 1] ['Pclass'].value_counts())
```

10. Get the number of non-survivors by class

```
print(data[data['Survived'] == 0] ['Pclass'].value_counts())
```

Screenshot:

The screenshot displays a web-based Python IDE interface for a course titled "4.2.7. Titanic Dataset Analysis and Data Cleaning - 3". The interface includes a sidebar with a list of 10 tasks, a main code editor, a terminal, and a test cases panel.

Tasks List:

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarked location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

Sample Data:

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. Owen Harris	male	22	0	0	A/5 21171	7.25	S	
2	1	1	Mr. John Bradley (Florence Briggs Thayer)	female	36	1	0	PC 17599	71.28	C85	C
3	1	3	Mr. James Heath (Miss May Platt)	female	26	0	0	STON/O 3101282	7.92	S	
4	1	1	Mr. Jacques Heath (Miss May Platt)	female	26	0	0	STON/O 3101282	7.92	S	

Code Editor:

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data['FamilySize'] = data['SibSp'] + data['Parch']
7 data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
8 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
9
10 # 1. Calculate the survival rate by class
11 print(data.groupby('Pclass')['Survived'].mean())
12
13 # 2. Calculate the survival rate by embarked location
14 print(data.groupby('Embarked_S')['Survived'].mean())
15
```

Test Cases:

Test case 1: Expected output: Pclass: 1... 0.629630, 2... 0.472826, 3... 0.242363. Actual output: Pclass: 1... 0.629630, 2... 0.472826, 3... 0.242363. Name: Survived, dtype: float64. Embarked_S: 0.629630, 0.472826, 0.242363.

4.2.8 titanic dataset analysis

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 1759
9,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O 2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C1
23,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11
.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

```

Note: Refer to the visible test case for better reference.

Program:

```
import pandas as pd
```

```
import numpy as np
```

Load the Titanic dataset

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
data = pd.get_dummies(data, columns=['Embarked'],
drop_first=True)
```

1. Get the number of survivors by gender

```
print(data[data['Survived'] == 1]['Sex'].value_counts())
```

2. Get the number of non-survivors by gender

```
print(data[data['Survived'] == 0]['Sex'].value_counts())
```

3. Get the number of survivors by embarked location

```
print(data[data['Survived'] ==  
1]['Embarked_S'].value_counts())
```

4. Get the number of non-survivors by embarked location

```
print(data[data['Survived'] ==  
0]['Embarked_S'].value_counts())
```

5. Calculate the percentage of children (Age < 18) who survived

```
children = data[data['Age'] < 18]  
print(children['Survived'].mean())
```

6. Calculate the percentage of adults (Age >= 18) who survived

```
adults = data[data['Age'] >= 18]
```

```
print(adults['Survived'].mean())
```

7. Get the median age of survivors

```
print(data[data['Survived'] == 1]['Age'].median())
```

8. Get the median age of non-survivors

```
print(data[data['Survived'] == 0]['Age'].median())
```

9. Get the median fare of survivors

```
print(data[data['Survived'] == 1]['Fare'].median())
```

10. Get the median fare of non-survivors

```
print(data[data['Survived'] == 0]['Fare'].median())
```

Screenshot:

Course

mitaoe.codetrantra.com/secure/course.jsp?euid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d5e/67e38...

CODETRANTRA Home Learn Anywhere

202501050040@mitaoe.ac.in Support Logout

4.2.6. Titanic Dataset Analysis and Data Cleaning - 4

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Brands, Mr. Owen Harris	male	22	1	0	A/5 21171	7.25	S	
2	1	1	Cummings, Mrs. John Bradley (Florence Briggs Thayer)	female	38	1	0	PC 17599	71.2833	C85	C
3	1	3	Hickson, Miss. Laina	female	26	0	0	STON/O2. 3101282	7.925	S	
4	1	1	Healy, Mr. Jacques Heath (Lily May Peel)	female	35	1	0	113803	53.1	C123	S

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
```

```
1,0,3,'Brands, Mr. Owen Harris',male,22,1,0,A/5 21171,7.25,S
```

```
2,1,1,'Cummings, Mrs. John Bradley (Florence Briggs Thayer)',female,38,1,0,PC 17599,71.2833,C85,C
```

```
3,1,3,'Hickson, Miss. Laina',female,26,0,0,STON/O2. 3101282,7.925,S
```

```
4,1,1,'Healy, Mr. Jacques Heath (Lily May Peel)',female,35,1,0,113803,53.1,C123,S
```

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
7
8
9 # 1. Get the number of survivors by gender
10 print(data[data['Survived'] == 1]['Sex'].value_counts())
11
12 # 2. Get the number of non-survivors by gender
13 print(data[data['Survived'] == 0]['Sex'].value_counts())
14
15 # 3. Get the number of survivors by embarkation location
```

Average time: 0.099 s
Maximum time: 99.00 ms
1 out of 1 shown test case(s) passed

Test case 1 99 ms DEBBUG

Expected output

```
female ... 233
male ... 109
```

Actual output

```
female ... 233
male ... 109
```

Name: Sex, dtype: int64

```
female ... 468
male ... 81
```

Name: Sex, dtype: int64

```
female ... 81
male ... 468
```

Name: Sex, dtype: int64

Terminal Test cases

PREV RESET SUBMIT NEXT

12:33 PM 04-05-2026

